

Class ImageOf

DETAILED DESCRIPTION

```
template<class T>
class yarp::sig::ImageOf< T >
```

Typed image class.

This is a wrapper over `yarp::sig::Image` providing type security for `pixel` access. "T" can be any of `sig::PixelMono`, `sig::PixelMono16`, `sig::PixelRgb`, `sig::PixelHsv`, `sig::PixelBgr`, `sig::PixelMonoSigned`, `sig::PixelRgbSigned`, `sig::PixelFloat`, `sig::PixelRgbFloat`, `sig::PixelHsvFloat`, `sig::PixelInt`. If `ImageOf::copy` is called for two such images, a reasonable casting operation will occur if the `pixel` types are different.

pixel() [1/2]

template<class T >

```
T& yarp::sig::ImageOf< T >::pixel ( size_t x,
                                   size_t y
                                   )
```

inline

Definition at line 663 of file Image.h.

width()

```
size_t yarp::sig::Image::width ( ) const
```

Gets width of image in pixels.

Returns

the width of the image in pixels (0 if no image present)

Examples

`imagemagick/main.cpp`, and `opencv/main.cpp`.

height()

```
size_t yarp::sig::Image::height ( ) const
```

Gets height of image in pixels.

Returns

the height of the image in pixels (0 if no image present)

Examples

`imagemagick/main.cpp`, and `opencv/main.cpp`.



Struct PixelRgb

yarp::sig::PixelRgb Struct Reference

Packed RGB pixel type. [More...](#)

```
#include <yarp/sig/Image.h>
```

PUBLIC MEMBER FUNCTIONS

PixelRgb ()=default

PixelRgb (unsigned char n_r, unsigned char n_g, unsigned char n_b)

PUBLIC ATTRIBUTES

unsigned char **r** {0}

unsigned char **g** {0}

unsigned char **b** {0}



How to read images

```
BufferedPort<ImageOf<PixelRgb> > imagePort;
```

```
ImageOf<PixelRgb> *image = imagePort.read();  
// read an image
```

```
if (image!=NULL) { // check if got something}
```

```
for (int x=0; x<image->width(); x++) {  
for (int y=0; y<image->height(); y++) {  
PixelRgb& pixel = image->pixel(x,y);  
if(pixel.b>...)
```

```
} } EXAMPLE: https://robotology.github.io/robotology-documentation/doc/html/findLocation\_8cpp\_source.html
```



Property class

Property is a class used to store configuration information. It is used to create an association between a keyword and its value.

Useful Member Functions:

put() [2/5]

```
void Property::put ( const std::string & key,  
                    const Value &    value  
                    )
```

Associate the given `key` with the given value.

After the association `find(key).asString()` will return that value. If `key` is already associated, the value will be replaced with the new one.

Parameters

`key` the key

`value` the value



Property class

Property is a class used to store configuration information. It is used to create an association between a keyword and its value.

Useful Member Functions:

◆ fromString()

```
void Property::fromString ( const std::string & txt,  
                           bool wipe = true  
                           )
```

Interprets a string as a list of properties.

Uses the the same format as a **Bottle**. The first element of every sub-list is interpreted as a key. For example, with text = `(width 10) (height 15)`, the **Property** object will be the mapping `{width => 10, height => 15}`. Therefore:

```
prop.find("width").asInt32() // gives 10  
prop.find("height").asInt32() // gives 15
```

If a key is duplicated, only the latest will be used. For example, with text = `(foo bar) (foo baz)`, the **Property** object will be the mapping `{foo => baz}`.

Parameters

txt the textual form of the **Property** object

wipe should **Property** be emptied first



Property class

Property is a class used to store configuration information. It is used to create an association between a keyword and its value.

Useful Member Functions:

check() [5/5]

bool yarp::os::Searchable::check

Gets a value corresponding to a given keyword.

If a property does not exist, this returns false and does not modify the result pointer. If a property exists but does not have a value, this again returns false and does not modify the result pointer.

```
Property p;
p.fromString("(width 10) (height 15) (help)");
p.check("help") // this is true
p.check("width") // this is true
p.check("foo") // this is false
Value *v;
p.check("help", v) // this is false, there is no value associated
p.check("width", v) // this is true, and v->asInt32() is 10
```

Parameters

- key** The keyword to look for
- result** A pointer to store the address of the result in
- comment** Optional human-readable explanation

Returns

True if there is a value corresponding to a given keyword, false otherwise. See the **find()** method for interpreting the value found.



Controlling the simulator

We set up a set of configuration parameters by using Properties:

```
Property options;  
options.put("device", "remote_controlboard");  
options.put("local", "/test/client");           //local port names  
options.put("remote", "/icubSim/right_arm");    //where we connect to
```

where the *local* and *remote* parameters are used to set the port names for the connection to the Simulator. This will create the following (local) ports:

```
/test/client/state:i  
/test/client/command:o  
/test/client/rpc:o
```

And will automatically connect them to the server, on the following (remote) ports

```
/icubSim/right_arm/state:o  
/icubSim/right_arm/command:i  
/icubSim/right_arm/rpc:i
```



PolyDriver

PolyDriver is a container for a device driver that implements Device Driver (an Interface implemented by the all device drivers). We need to get a Pointer to the driver we want to use. We will use the *view()* member function

◆ view()

template<class T >

```
bool yarp::dev::DeviceDriver::view ( T*& x )
```

Get an interface to the device driver.

Parameters

x A pointer of type T which will be set to point to this object if that is possible.

Returns

true iff the desired interface is implemented by the device driver.



PolyDriver

PolyDriver is a container for a device driver that implements Device Driver (an Interface implemented by the all device drivers). We need to get a *pointer* to the driver we want to use. We will use the *view()* member function

```
PolyDriver robotDevice(options);  
if (!robotDevice.isValid()) {  
    ACE_OS::printf("Device not available. Here are the known devices:\n");  
    ACE_OS::printf("%s", Drivers::factory().toString().c_str());  
    Network::fini();  
    return 1;  
}
```

```
IPositionControl *pos;  
IVelocityControl *vel;  
IEncoders *enc;
```

```
robotDevice.view(pos);  
robotDevice.view(vel);  
robotDevice.view(enc);
```

get a set of interfaces
(pointers to) to work with



IpositionControl class

It is an interface for a control board device that implements position control.

◆ setRefSpeeds() (1/2)

```
virtual bool yarp::dev::IPositionControl::setRefSpeeds ( const double * spds )
```

Set reference speed on all joints.

These values are used during the interpolation of the trajectory.

Parameters

spds pointer to the array of speed values.

Returns

true/false upon success/failure

◆ setRefAccelerations() (1/2)

```
virtual bool yarp::dev::IPositionControl::setRefAccelerations ( const double * accs )
```

Set reference acceleration on all joints.

This is the value that is used during the generation of the trajectory.

Parameters

accs pointer to the array of acceleration values

Returns

true/false upon success/failure



IpositionControl class

It is an interface for a control board device that implements position control.

◆ positionMove() [1/3]

```
virtual bool yarp::dev::IPositionControl::positionMove ( const double * refs )
```

Set new reference point for all axes.

Parameters

refs array, new reference points.

Returns

true/false on success/failure

◆ positionMove() [2/3]

```
virtual bool yarp::dev::IPositionControl::positionMove ( const int n_joint,  
                                                         const int * joints,  
                                                         const double * refs  
                                                         )
```

Set new reference point for a subset of joints.

Parameters

joints pointer to the array of joint numbers

refs pointer to the array specifying the new reference points

Returns

true/false on success/failure



IEncoders

Encoder interface of a control board device

◆ getEncoder()

```
virtual bool yarp::dev::IEncoders::getEncoder ( int      j,  
                                                double * v  
                                                )
```

Read the value of an encoder.

Parameters

- j** encoder number
- v** pointer to storage for the return value

Returns

true/false, upon success/failure (you knew it, uh?)

◆ getEncoders()

```
virtual bool yarp::dev::IEncoders::getEncoders ( double * encs )
```

Read the position of all axes.

Parameters

encs pointer to the array that will contain the output

Returns

true/false on success/failure



Control the simulator

```
Vector tmp;  
    Vector encoders;  
    Vector command_position;  
    Vector command_velocity;  
    tmp.resize(jnts);  
    encoders.resize(jnts);  
  
enc->getEncoders(encoders.data()); //encoder reading  
for (i = 0; i < jnts; i++) {  
    tmp[i] = 40.0;  
}  
pos->positionMove(tmp.data());
```



Exercise

Make a program that receives input from the user and move the head. Check if the position has been reached by the joint before receiving a new one

